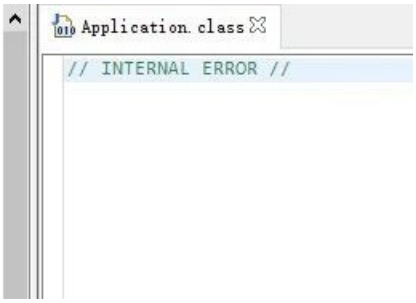


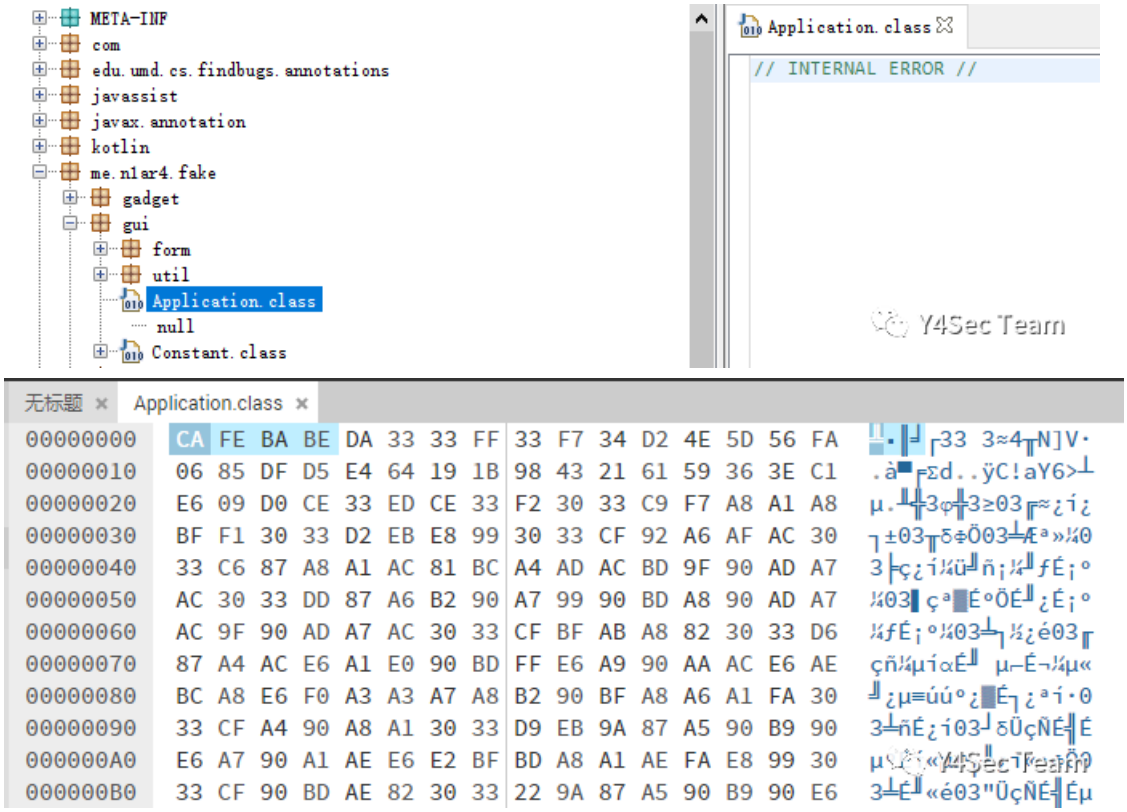
JVMTI 加密字节码详解

Y4Sec Team Y4Sec Team

0x00 介绍

这不是新思路，但网上的文章不够深入和详细，因此有了这篇文章
上周偶然看到一篇文章
https://juejin.cn/post/6844903487784894477
以及 Github 仓库代码
https://github.com/sea-boat/ByteCodeEncrypt
感觉是一个很有趣的项目，对于 Jar 包以及 Class 的保护通常是使用 ProGuard 等工具，对 Class 文件本身的混淆。而该文章提到了一种更巧妙的办法，总体来说是以下的思路：
1. 用 C 编写加密算法，调用 JNI 加密指定类名的 Class 文件并保存
2. 启动 JVM 时利用 JVMTI 在加载 Class 文件时解密

参考原 作者文章： 利用JDK中JVM的某些类似钩子机制和事件监听机制，监听加载 Class 事件，使用本地方式完成 Class 的解密。 C/C++ 被编译后想要反编译就很麻烦了，另外还能加壳
可以看到加密后的 Class 文件不是合法字节码文件（开头魔数做了特殊处理，让这个文件看起来是 Class 文件）



原文章和原项目有一些小问题：
1. 原文章固定了包名，用户想加密自己的包名需要 **重新编译** DLL

2. 原文章加密和解密 DLL 是 **同一个**，这样只用 JNI 调用下加密即可解 决
3. 原文章的代码仅是 **Demo** 级别，无法直接上手测试和使用
4. 原文章没有加入具体的 **加密算法**，仅是简单的运算，需要加强
5. 原文章的代码存在一些 BUG 和 **优化空间**
6. 补充：原文章没有提到这种加密如何绕过

这个思路很有意思，于是我打算深入研究下，在原作者文章基础上做一些详细的补充，并且写一些代码，尝试做一个可以直接使用的工具

0x01 JVMTI

这里我们先看一下 JVMTI 的功能，学新技术最好的办法是看官方文档
https://docs.oracle.com/javase/8/docs/platform/jvmti/jvmti.html
官方文档较长，参考原作者的介绍：
JVMTI即JVM Tool Interface，提供了本地编程接口，主要是提供了调试和分析等接口。JVMTI非常强大，通过它能做很多事，比如可以监听某事件、线程分析等等。 一般使用Agent方式来使用，就是通过-agentlib和-agentpath指定Agent的本地库，然后Java启动时就会加载该动态库。这个时刻可以看成是JVM启动的时刻，而并非是Java层程序启动时刻，所以此时不涉及与Java相关的类和对象什么的。

Agent Start-Up (OnLoad phase)

If an agent is started during the `OnLoad` phase then its agent library must export a start-up function with the following prototype:

```
JNIEXPORT jint JNICALL
Agent_OnLoad(JavaVM *vm, char *options, void *reserved)
```

Y4Sec Team

简单来说，如果是启动阶段的 Agent 必须导出 Agent_Onload

The VM will call the `Agent_OnLoad` or `Agent_OnLoad_<agent-lib-name>` function with *<options>* as the second argument - that is, using the command-line option examples, `"opt1,opt2"` will be passed to the `char *options` argument of `Agent_OnLoad`. The `options` argument is encoded as a modified UTF-8 string. If *=<options>* is not specified, a zero length string is passed to `options`. The

Y4Sec Team

注意到我们是可以通过 agentlib 传递参数的，这里指出参数通过 char *options 传递，这个点的意义在于：可以传入具体的包名，决定通过 JVMTI 解密的包名是什么
注意到下文有一个 Agent_OnAttach 函数，为什么不使用该函数？这个函数是用于 Attach 功能实时代理，也就是 Java Agent 所说的动态 Agent 类型。我们修改字节码显然不可以是动态的，而是需要在启动时处理
文档提出三种字节码检测操作，第一种方式官方的形容是 extremely awkward 忽略不考虑，第二种是加载时检测，第三种是动态监测，看起来是动态 Java Agent 的 RetransformClass 功能。我们的需要正好符合了第二条加载时检测： **在 JVM 真正加载 Class 之前，把 Class 文件解密了**

- Load-Time Instrumentation: When a class file is loaded by the VM, the raw bytes of the class file are sent for instrumentation to the agent. The `ClassFileLoadHook` event, triggered by the class load, provides this functionality. This mechanism provides efficient and complete access to one-time instrumentation.
- Dynamic Instrumentation: A class which is already loaded (and possibly even running) is modified. This optional feature is provided by the `ClassFileLoadHook` event, triggered by calling the `RetransformClasses` function. Classes can be modified multiple times and can be returned to their original state. The mechanism allows instrumentation which changes during the course of execution.

Y4Sec Team

文档提到需要关注的事件是 ClassFileLoadHook
当 VM 获取类文件数据时但在构造该类的内存中表示之前发送此事件

> The agent must allocate the space for the modified class file data buffer using the memory allocation function Allocate because the VM is responsible for freeing the new class file data buffer using Deallocate. Note that Allocate is permitted during the primordial phase.
>
> If the agent wishes to modify the class file, it must set new_class_data to point to the newly instrumented class file data buffer and set new_class_data_len to the length of that buffer before returning from this call. If no modification is desired, the agent simply does not set new_class_data. If multiple agents have enabled this event the results are chained. That is, if new_class_data has been set, it becomes the class_data for the next agent.

参考上图这个点需要重点关注两件事:

1. 必须使用 allocate 函数为修改后的类文件数据缓冲区分配空间
2. 如果修改类文件必须修改 new_class_data 指向新 buffer

ClassFileLoadHook 对应的事件如下图，使用函数 SetEventNotificationMode 即可使事件通知生效

Phase	Event Type	Number	Enabling	Since
sent during the primordial, start or live phase	JVMTI_EVENT_CLASS_FILE_LOAD_HOOK	54	SetEventNotificationMode(JVMTI_ENABLE, JVMTI_EVENT_CLASS_FILE_LOAD_HOOK, NULL)	1.0

另外 JVMTI 有一个重要结构叫做 jvmtiCapabilities （能力）
大致意思为每个 JVMTI 可以添加具体的功能，官方提到不建议开启全部的功能，因为可能会因其未使用的功能而遭受性能损失

> For example, a freshly started agent (in the OnLoad function) wants to enable all possible capabilities. Note that, in general, this is not advisable as the agent may suffer a performance penalty for functionality it is not using. The code might look like this in C:

原文代码开启了多个功能，实际上翻阅功能文档，我们需要的只是以下

Capabilities	
Required Functionality	
Optional Features	
Capability	Effect
can_generate_all_class_hook_events	Can generate ClassFileLoadHook events for every loaded class.

翻译为代码则是

```
capabilities.can_generate_all_class_hook_events = 1;
```

读到这里，我们以及了解了如何通过 JVMTI 解密字节码了

0x02 JVMTI 代码

有了上一章的内容，现在我们编写一个 agentlib dll 库将会很简单
在代码仓库的 start.c 文件中，开头 50 行可能看起来复杂，其实功能很简单。拿到 options 数据，根据 = 号分割，替换包名中的 . 为 / 符号。不得不说，C 语言写这样简单的一个逻辑都得几十行，Go/Java 可能只用几行
第一步初始化 JVMTI

```
jint ret = (*vm)->GetEnv(vm, (void **) &jvmti, JVMTI_VERSION);
```

这里的需要用 JVMTI_VERSION 变量，这个版本是导致不同 JVM 版本无法兼容 JVMTI DLL 文件的原因，例如 JDK-11 和 JDK-8 的 JNI.h 头文件中，这个值不一样。如果你想做不同 JDK 版本的库，需要另外编译
第二步设置 JVMTI 能力

```
LOG("INIT JVMTI CAPABILITIES");
jvmtiCapabilities capabilities;
(void) memset(&capabilities, 0, sizeof(capabilities));

capabilities.can_generate_all_class_hook_events = 1;

LOG("ADD JVMTI CAPABILITIES");
jvmtiError error = (*jvmti)->AddCapabilities(jvmti, &capabilities);
if (JVMTI_ERROR_NONE != error) {
    printf("ERROR: Unable to AddCapabilities JVMTI!\n");
    return error;
}
```

上文讨论过，我们 ClassFileLoadHook 功能仅需要开启 can_generate_all_class_hook_events 能力，其他能力开启会消耗性能
第三步设置回调
调用JVMTI提供的SetEventCallbacks函数，向JVMTI注册事件回调
这里的 ClassDecryptHook 函数后续介绍

```
LOG("INIT JVMTI CALLBACKS");
jvmtiEventCallbacks callbacks;
(void) memset(&callbacks, 0, sizeof(callbacks));

LOG("SET JVMTI CLASS FILE LOAD HOOK");
callbacks.ClassFileLoadHook = &ClassDecryptHook;
error = (*jvmti)->SetEventCallbacks(jvmti, &callbacks, sizeof(callbacks));
if (JVMTI_ERROR_NONE != error) {
    printf("ERROR: Unable to SetEventCallbacks JVMTI!\n");
    return error;
}
```

第四步设置事件通知模式
上文已经提到 ClassFileLoadHook 需要设置某个 Event 后才会生效，对应的代码如下

```
error = (*jvmti)->SetEventNotificationMode(jvmti, JVMTI_ENABLE, JVMTI_EVENT_CLASS_FILE_LOAD_HOOK, NULL
);
if (JVMTI_ERROR_NONE != error) {
    printf("ERROR: Unable to SetEventNotificationMode JVMTI!\n");
    return error;
}

LOG("INIT JVMTI SUCCESS");
```

最后一步是 ClassFileLoadHook 回调
函数定义和两处关键点参考 JVMTI 文档：使用 Allocate 分配内存；修改 new_class_data 指向的 buffer 内容。_data 是指向 new_class_data 的指针。当你修改 _data 所指向的内存地址的内容时，new_class_data 会发生变化，因为它们指向同一个内存地址
如果包名匹配到传入的参数，才会进行解密处理，否则正常执行

```
void JNICALL ClassDecryptHook(
        jvmtiEnv *jvmti_env,
        JNIEnv *jni_env,
        jclass class_being_redefined,
        jobject loader,
        const char *name,
        jobject protection_domain,
        jint class_data_len,
        const unsigned char *class_data,
        jint *new_class_data_len,
        unsigned char **new_class_data) {
    *new_class_data_len = class_data_len;
    (*jvmti_env)->Allocate(jvmti_env, class_data_len, new_class_data);
    unsigned char *_data = *new_class_data;
    if (name && strncmp(name, PACKAGE_NAME, strlen(PACKAGE_NAME)) == 0) {
        for (int i = 0; i < class_data_len; i++) {
            _data[i] = class_data[i];
        }
        // ...
        decrypt((unsigned char *) _data, class_data_len);
    } else {
        for (int i = 0; i < class_data_len; i++) {
            _data[i] = class_data[i];
        }
    }
}
```

0x03 加密解密代码

以上已经有了 JVMTI 解密部分的核心代码，还差具体的加密解密代码
使用 DES/AES 是一种办法，但是使用 C 实现起来比较复杂，也可以考虑使用 OpenSSL 来做，笔者这里抛砖引玉，具体加密解密可以自行发挥
我选择的加密解密算法是：XXTEA 算法 结合 位运算加密
选择 XXTEA 算法由于其 C 实现代码比较简单，且有一定的强度

```c
void tea_encrypt(uint32_t *v, const uint32_t *k) {
    uint32_t v0 = v[0], v1 = v[1], sum = 0, i;
    uint32_t delta = 0x9e3779b9;
    uint32_t k0 = k[0], k1 = k[1], k2 = k[2], k3 = k[3];
    for (i = 0; i < 32; i++) {
        sum += delta;
        v0 += ((v1 << 4) + k0) ^ (v1 + sum) ^ ((v1 >> 5) + k1);
        v1 += ((v0 << 4) + k2) ^ (v0 + sum) ^ ((v0 >> 5) + k3);
    }
    v[0] = v0;
    v[1] = v1;
}

void tea_decrypt(uint32_t *v, const uint32_t *k) {
    uint32_t v0 = v[0], v1 = v[1], sum = 0xC6EF3720, i;
    uint32_t delta = 0x9e3779b9;
    uint32_t k0 = k[0], k1 = k[1], k2 = k[2], k3 = k[3];
    for (i = 0; i < 32; i++) {
        v1 -= ((v0 << 4) + k2) ^ (v0 + sum) ^ ((v0 >> 5) + k3);
        v0 -= ((v1 << 4) + k0) ^ (v1 + sum) ^ ((v1 >> 5) + k1);
        sum -= delta;
    }
    v[0] = v0;
    v[1] = v1;
}
```

以上代码来源于网上博客，加密的要求是：输入两个 32 位数，提供四个 32 位数作为密钥，加密得到两个 32 位数
具体的加密还需要特殊处理：需要想办法把 char* 转为 int32 类型，加密完需要把 int32 类型转回 char* 再写入原 chars
具体的 convert 和 revert 函数参考代码仓库，主要是一些位运算
这里暂固定密钥 Y4Sec-Team-4ra1n（这个可以由 options 参数传入）

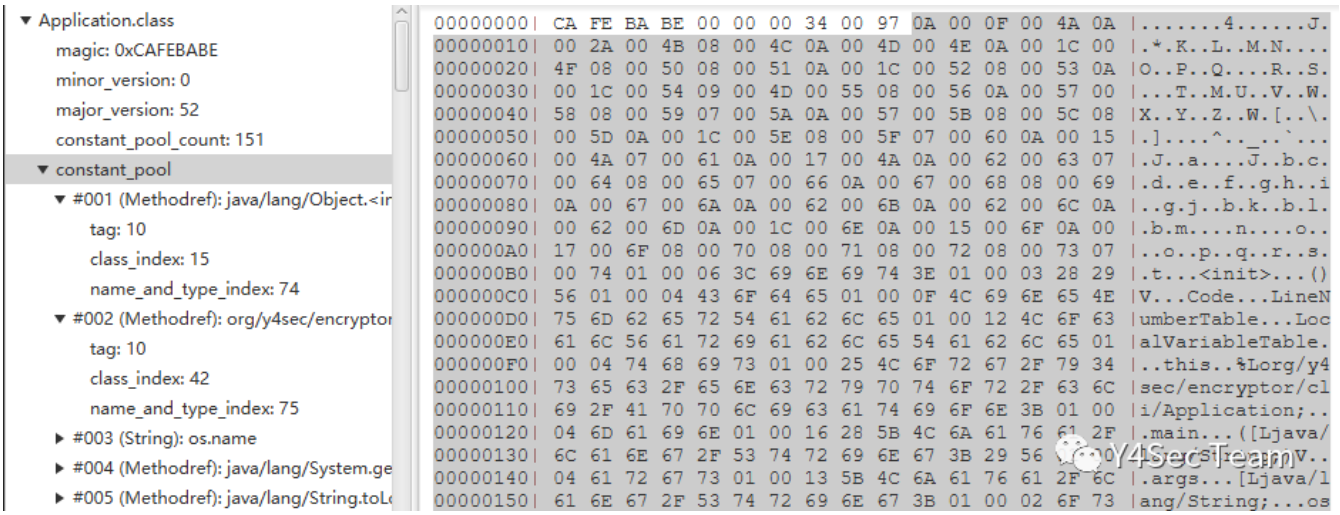
```
void internal(unsigned char *chars, int start) {
    unsigned char first[4];
    for (int i = start; i < start + 4; i++) {
        first[i - start] = chars[i];
    }
    unsigned char second[4];
    for (int i = start + 4; i < start + 8; i++) {
        second[i - start - 4] = chars[i];
    }
    uint32_t v[2] = {convert(first), convert(second)};
    // key: Y4Sec-Team-4ra1n
    // 59345365 632D5465 616D2D34 7261316E
    uint32_t const k[4] = {
            (unsigned int) 0x65533459, (unsigned int) 0x65542d63,
            (unsigned int) 0X342d6d61, (unsigned int) 0x6e316172,
    };
    tea_encrypt(v, k);
    unsigned char first_arr[4];
    unsigned char second_arr[4];
    revert(v[0], first_arr);
    revert(v[1], second_arr);
    for (int i = start; i < start + 4; i++) {
        chars[i] = first_arr[i - start];
    }
    for (int i = start + 4; i < start + 8; i++) {
        chars[i] = second_arr[i - start - 4];
    }
}
```

按照网上这份 XXTEA 加密代码来看，一次 XXTEA 加密的数据仅 8 个字节，想要完全使用 XXTEA 加密，需要按照每次 8 个的顺序循环加密字节码。这里我仅选择字节码关键部分，加密三组 24 字节。读者可以自行拓展，从这里直接加密到结束，完全加密

关键部分的选择我考虑从第 10 个字节开始，加密到第 32 个字节

参考 ClassFile 结构，magic/version/count 部分不包含敏感信息，没有必要进行加密，从第 10 个字节开始常量池包含了敏感信息，以此开始

```
// ClassFile {
//     u4             magic; (ignore)
//     u2             minor_version; (ignore)
//     u2             major_version; (ignore)
//     u2             constant_pool_count; (ignore)
//     cp_info        constant_pool[constant_pool_count-1];
//     ...
// }
```



于是 JNI 加密的代码如下

```
JNIEXPORT jbyteArray JNICALL Java_org_y4sec_encryptor_core_CodeEncryptor_encrypt
        (JNIEnv *env, jclass cls, jbyteArray text, jint length) {
    jbyte *data = (*env)->GetByteArrayElements(env, text, NULL);
    unsigned char *chars = (unsigned char *) malloc(length);
    memcpy(chars, data, length);
    // 1. asm encrypt
    encrypt(chars, length);
    LOG("ASM ENCRYPT FINISH");
    // 2. tea encrypt
    if (length < 34) {
        LOG("ERROR: BYTE CODE TOO SHORT");
        return text;
    }
    // {[10:14],[14:18]}
    internal(chars, 10);
    LOG("TEA ENCRYPT #1");
    // {[18:22],[22:26]}
    internal(chars, 18);
    LOG("TEA ENCRYPT #2");
    // {[26:30],[30:34]}
    internal(chars, 26);
    LOG("TEA ENCRYPT #3");
    (*env)->SetByteArrayRegion(env, text, 0, length, (jbyte *) chars);
    return text;
}
```

代码第一步我先对完整字节码做了位运算加密

```
// 1. asm encrypt
encrypt(chars, length);
LOG("ASM ENCRYPT FINISH");
```

这部分代码由汇编编写，核心部分如下
- 遍历 char* 字节码，对每一位进行位运算
- 位运算主要包含多次抑或，加减，非操作

```
link_start:
    ; if rbx >= rcx goto end
    cmp rbx, rcx
    jge magic
    ; al = str[rdi+rbx]
    mov al, byte ptr [rdi+rbx]
    ; al = al - 2
    sub al, 002h
    ; al = al ^ 11h
    xor al, 011h
    ; al = ~al
    not al
    ; al = al + 1
    add al, 001h
    ; al = al ^ 22
    xor al, 022h
    ; str[rdi+rbx] = al
    mov byte ptr [rdi+rbx], al
    ; ebx ++
    inc rbx
    ; loop
    jmp link_start
```

位运算结束后，我特殊处理了 MAGIC 头，使开头按照 JAVA MAGIC 的 CAFE BABE 格式，起到一定程度的混淆

```
magic:
    ; magic
    mov al, 0CAh
    mov byte ptr [rdi+000h], al
    mov al, 0FEh
    mov byte ptr [rdi+001h], al
    mov al, 0BAh
    mov byte ptr [rdi+002h], al
    mov al, 0BEh
    mov byte ptr [rdi+003h], al
```

在结束时，我做了最后一层加密：取第4位直接和末尾字节交换。简单的字节交换也会导致字节码无法执行和解析报错

```
; signature
mov rsi, rcx
sub rsi, 001h
mov al, byte ptr [rdi+rsi]
mov ah, byte ptr [rdi+004h]
mov byte ptr [rdi+004h], al
mov byte ptr [rdi+rsi], ah
; reset
xor ah, ah
xor al, al
xor rsi, rsi
```

0x04 工程化

加密代码如上，解密代码只要你过来即可，可以参考代码仓库，这里不再提及了。接下来我们看 Java 层的代码，逻辑很简单，读取输入 Jar 包，其中匹配到我们期望 PACKAGE NAME 的类调用 JNI 加密方法进行加密，然后把结果写入新的 Jar 包即可

```java
// ...
while (enumeration.hasMoreElements()) {
    JarEntry entry = enumeration.nextElement();
    InputStream is = srcJar.getInputStream(entry);
    int len;
    while ((len = is.read(buf, 0, buf.length)) != -1) {
        bao.write(buf, 0, len);
    }
    byte[] bytes = bao.toByteArray();

    String name = entry.getName();
    if (name.startsWith(packageName)) {
        if (name.toLowerCase().endsWith(ClassFile)) {
            try {
                bytes = CodeEncryptor.encrypt(bytes, bytes.length);
            } catch (Exception e) {
                logger.error("encrypt error: {}", e.toString());
                return;
            }
        }
    }
    // ...
}
```

Java 的 JNI 加载其实是有一些坑的，比如 System.loadLibrary 方法是不支持绝对路径的，只能从系统 lib 以及当前目录加载。这里 JNIUtil.java 我做了一些魔法操作，使得可以加载任意路径的 dll 文件
最终使用一个简单的命令即可加密 Jar 包

```
java -jar code-encryptor-plus.jar patch --jar your-jar.jar --package com.your.pack
```

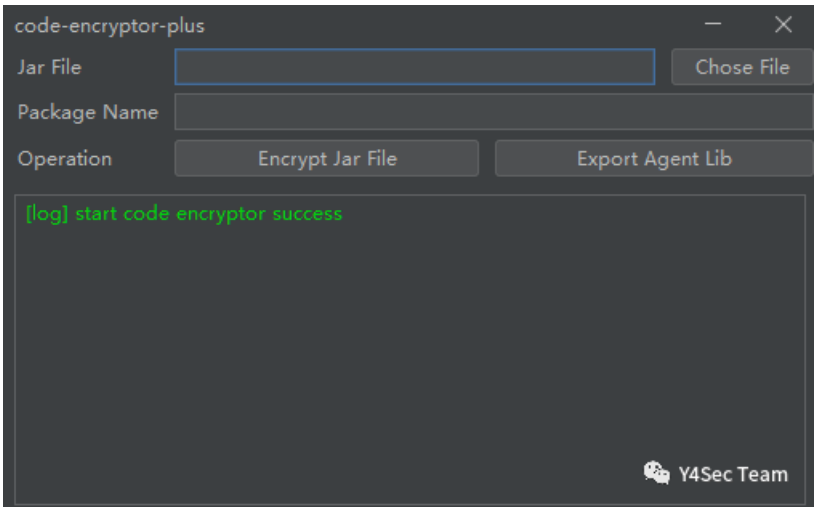
导出解密 DLL 文件：（默认导出到code-encryptor-plus-temp目录）

```
java -jar code-encryptor-plus.jar export
```

使用解密DLL启动Jar包：（使用-agentlib参数）
由于 agentlib 的特性，要求必须是绝对路径的 DLL 且结尾去掉 DLL 才可

```
java -agentlib:D:\abs-path\decrypter=PACKAGE_NAME=com.your.pack --jar your-jar.jar
```

另外支持了简易的GUI版本，选择需要加密的Jar文件即可一键加密



简单实践一下，加密我自己写的 Fake MySQL Server

```
java -jar .\code-encryptor-plus-0.0.1-cli.jar patch --jar .\fake-mysql-gui-0.0.3.jar --package me.n1ar4
```

如果直接使用 java -jar 启动加密后的 jar 包，报错

```
PS C:\JavaCode\code-encryptor-plus\target> java -jar .\fake-mysql-gui-0.0.3_encrypted.jar
Error: A JNI error has occurred, please check your installation and try again
Exception in thread "main" java.lang.UnsupportedClassVersionError: me/n1ar4/fake/gui/Application has been compiled by
.55859), this version of the Java Runtime only recognizes class file versions up to 52.0
        at java.lang.ClassLoader.defineClass1(Native Method)
        at java.lang.ClassLoader.defineClass(Unknown Source)
        at java.security.SecureClassLoader.defineClass(Unknown Source)
        at java.net.URLClassLoader.defineClass(Unknown Source)
        at java.net.URLClassLoader.access$100(Unknown Source)
        at java.net.URLClassLoader$1.run(Unknown Source)
        at java.net.URLClassLoader$1.run(Unknown Source)
        at java.security.AccessController.doPrivileged(Native Method)
        at java.net.URLClassLoader.findClass(Unknown Source)
        at java.lang.ClassLoader.loadClass(Unknown Source)
        at sun.misc.Launcher$AppClassLoader.loadClass(Unknown Source)
        at java.lang.ClassLoader.loadClass(Unknown Source)
        at sun.launcher.LauncherHelper.checkAndLoadMain(Unknown Source)
PS C:\JavaCode\code-encryptor-plus\target>
```

Y4Sec Team

导出解密 dll 文件

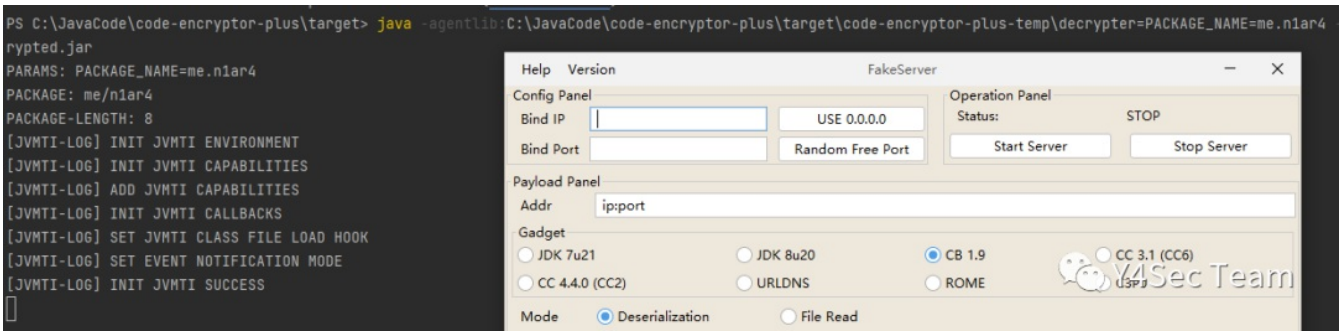
```
java -jar .\code-encryptor-plus-0.0.1-cli.jar export
```

```
PS C:\JavaCode\code-encryptor-plus\target> java -jar .\code-encryptor-plus-0.0.1-cli.jar export
-----------                                   __
\_   _____/ ____   ___________ ___.__._______/  |_  ___________
 |    __)_ /    \_/ ___\_  __ <   |  |\____ \   __\/  _ \_  __ \
 |        \   |  \  \___|  | \/\___  ||  |_> >  | (  <_> )  | \/
/_______  /___|  /\___  >__|   / ____||   __/|__|  \____/|__|
        \/     \/     \/       \/     |__|
Bytecode Encryptor Tool - version: 0.0.1
Powered By Y4Sec-Team
14:10:23 [INFO] [ExportCommand] execute export command
----------- ADD VM OPTIONS -----------
java -agentlib:path-to-file=PACKAGE_NAME=xxx [other-params]
java -agentlib:encryptor=PACKAGE_NAME=org.y4sec -jar test.jar
PS C:\JavaCode\code-encryptor-plus\target>
```

Y4Sec Team

使用 agentlib 加载解密 dll 文件启动 jar 包，成功启动

```
java -agentlib:C:\JavaCode\code-encryptor-plus\target\code-encryptor-plus-temp\decrypter=PACKAGE_NAME=
me.n1ar4 -jar .\fake-mysql-gui-0.0.3_enc
rypted.jar
```



0x05 拓展

如何破解这种加密呢

对于位加密来说，通过 x64dbg 手动调试看汇编是一种办法

```
00007FF6916D110B   8A0437        MOV AL,BYTE PTR DS:[RDI+RSI]
00007FF6916D110E   8A67 04       MOV AH,BYTE PTR DS:[RDI+4]
00007FF6916D1111   8847 04       MOV BYTE PTR DS:[RDI+4],AL
00007FF6916D1114   882437        MOV BYTE PTR DS:[RDI+RSI],AH
00007FF6916D1117   32E4          XOR AH,AH
00007FF6916D1119   32C0          XOR AL,AL
00007FF6916D111B   48:33F6       XOR RSI,RSI
00007FF6916D111E   48:3BD9       CMP RBX,RCX
00007FF6916D1121   7D 14         JGE decrypt_test.7FF6916D1137
00007FF6916D1123   8A041F        MOV AL,BYTE PTR DS:[RDI+RBX]
00007FF6916D1126   34 22         XOR AL,22
00007FF6916D1128   2C 01         SUB AL,1
00007FF6916D112A   F6D0          NOT AL
00007FF6916D112C   34 11         XOR AL,11
00007FF6916D112E   04 02         ADD AL,2
00007FF6916D1130   88041F        MOV BYTE PTR DS:[RDI+RBX],AL
00007FF6916D1133   FFC3          INC EBX
00007FF6916D1135   EB E7         JMP decrypt_test.7FF6916D111E
```

Y4Sec Team

通过 IDA F 5 能看到更友好的代码

```
char __fastcall sub_1400010F0(__int64 a1, signed __int64 a2)
{
  signed __int64 v2; // rbx
  char v3; // ah
  char result; // al

  v2 = 4i64;
  v3 = *(_BYTE *)(a1 + 4);
  *(_BYTE *)(a1 + 4) = *(_BYTE *)(a1 + a2 - 1);
  *(_BYTE *)(a1 + a2 - 1) = v3;
  result = 0;
  while ( v2 < a2 )
  {
    result = (~((*(_BYTE *)(a1 + v2) ^ 0x22) - 1) ^ 0x11) + 2;
    *(_BYTE *)(a1 + v2) = result;
    v2 = (unsigned int)(v2 + 1);
  }
  return result;
}
```

Y4Sec Team

位加密可以很简单的破解

而 XXTEA 加密会稍微复杂一些

```
v4 = -957401312;
v5 = *a2;
v6 = 32i64;
v7 = a2[1];
v8 = a2[2];
v9 = a2[3];
do
{
  result = (unsigned int)result - ((v4 + v2) ^ (v8 + 16 * v2) ^ (v9 + (v2 >> 5)));
  v10 = (v4 + result) ^ (v5 + 16 * result) ^ (v7 + ((unsigned int)result >> 5));
  v4 += 1640531527;
  v2 -= v10;
  --v6;
}
while ( v6 );
*a1 = v2;
a1[1] = result;
return result;
```

Y4Sec Team

可以动态的方式拿到密钥，结合上面的算法进行解密

```
MOV QWORD PTR SS:[RSP+8],RBX
MOV QWORD PTR SS:[RSP+10],RBP
MOV QWORD PTR SS:[RSP+18],RSI
MOV QWORD PTR SS:[RSP+20],RDI
MOV R8D,DWORD PTR DS:[RCX]
MOV R9,RCX
MOV EAX,DWORD PTR DS:[RCX+4]
MOV R10D,C6EF3720
MOV EBX,DWORD PTR DS:[RDX]
MOV R11D,20                                  20:' '
MOV EDI,DWORD PTR DS:[RDX+4]
MOV ESI,DWORD PTR DS:[RDX+8]
MOV EBP,DWORD PTR DS:[RDX+C]
NOP DWORD PTR DS:[RAX],EAX
NOP DWORD PTR DS:[RAX+RAX],EAX
MOV EDX,R8D
MOV ECX,R8D
SHR EDX,5
```

```
隐藏FPU
RAX   0000000061E0801A
RBX   0000000065533459
RCX   0000009E22DFFBA8
RDX   0000009E22DFFBB8
RBP   000000006E316172
RSP   0000009E22DFFB78
RSI   00000000342D6D61
RDI   0000000065542D63

R8    00000000E20818D5
R9    0000009E22DFFBA8
R10   00000000C6EF3720
R11   0000000000000020
R12   0000000000000000
R13   0000000000000000
```

Y4Sec Team

逆向来做是走了弯路，另有路子解决这种办法
使用 sa-jdi.jar 的 HSDB 即可

```
java -cp "C:\Program Files\Java\jdk1.8.0_131\lib\sa-jdi.jar" sun.jvm.hotspot.HSDB
```

使用以下方式即可拿到 Class

Class Browser

me.n1ar4

Create .class for all classes

public abstract interface com.formdev.flatlaf.FlatClientProperties @0x00000007c0142af8

public abstract class com.formdev.flatlaf.FlatDefaultsAddon @0x00000007c00b8d10

public final [enum] class com.formdev.flatlaf.FlatIconColors [signature Ljava/lang/Enum<Lcom/formdev/flatlaf/FlatIconColors;>;] @0x00000007c00b82e0

class com.formdev.flatlaf.FlatInputMaps @0x00000007c00b86f0

class com.formdev.flatlaf.FlatInputMaps$LazyInputMapEx @0x00000007c00b8af8

class com.formdev.flatlaf.FlatInputMaps$LazyModifyInputMap @0x00000007c00b88e0

public class com.formdev.flatlaf.FlatIntelliJLaf @0x00000007c00bcb80

public abstract class com.formdev.flatlaf.FlatLaf @0x00000007c0060218

class com.formdev.flatlaf.FlatLaf$ActiveFont @0x00000007c00b80b8

class com.formdev.flatlaf.FlatLaf$FlatUIDefaults @0x00000007c00612f0

public class com.formdev.flatlaf.FlatLightLaf @0x00000007c0060828

public class me.n1ar4.fake.gui.Application @0x00000007c0060028

View Class Hierarchy

Create .class File

Super Class

public class java.lang.Object @0x00000007c0000f28

Methods

- public void <init>() @0x0000000024060320;
- public static void main(java.lang.String[]) @0x00000000240603d0;
- private static void createAndShowGUI() @0x0000000024060468;

Y4Sec Team

Constant Pool

sa-jdi.jar 默认会把 class 保存到 C:\User 里

```
piled .class file, bytecode version: 52.0 (Java 8)
//
// Source code recreated from a .class file by IntelliJ IDEA
// (powered by FernFlower decompiler)
//

package me.n1ar4.fake.gui;

import ...

public class Application {
    public Application() {
    }

    public static void main(String[] args) {
        FlatLightLaf.setup();
        SwingUtilities.invokeLater( doRun: run<invokedynamic>());
    }

    no usages
```

最终成功拿到了代码
HSDB 的缺点是:
- 程序完全卡死，这对于某些业务来说是不可取的行为
- 只能拿到部分 Class 而不是所有的 Class
HSDB只能查看已经加载到JVM中的类，如果某个类尚未被加载，HSDB将无法访问该类的调试信息

0x06 总结

这种加密方式是比较有意思的，我在原作者的基础上，做了进一步的拓展，详细讲解了 JVMTI 部分的代码。其中加密算法部分也是抛砖引玉，使用经典的 XXTEA 算法和位加密。其中还有进一步拓展的地方：比如汇编加密解密算法部分加入花指令，让逆向人员头疼；比如 DLL 可以使用 OLLVM/VMP 混淆
完整的项目代码在
https://github.com/Y4Sec-Team/code-encryptor-plus